```
fe.create_table(
    name="lakehouse_in_action.database_name.
    feature_table_name_ft",
    primary_keys=["sequence_id","sequence_step"],
    df=feature_dataframe,
    description="This feature table was created by
    x,y,z and would be great for augmenting w for
    modeling or business use case q.")
```

Figure 5.1 – Example of creating a feature table with FeatureEngineeringClient

As we mentioned, the combination of Delta, UC, and the Feature Engineering client provides considerable benefits. Features can be shared and reused across teams, reducing the need to recreate them from scratch. Reusability can save time and resources, and it can also help to ensure that features are consistent across different teams. Centralized feature tables ensure the same code computes feature values for training and inference. This is especially important to avoid online/offline skew – that is, the discrepancy that can occur when the data transformations used during the model training phase (offline) are different from those used when the model is deployed and making predictions (online). *Figure 5.2* shows a data science project workflow and highlights the offline and online transformations that must match.

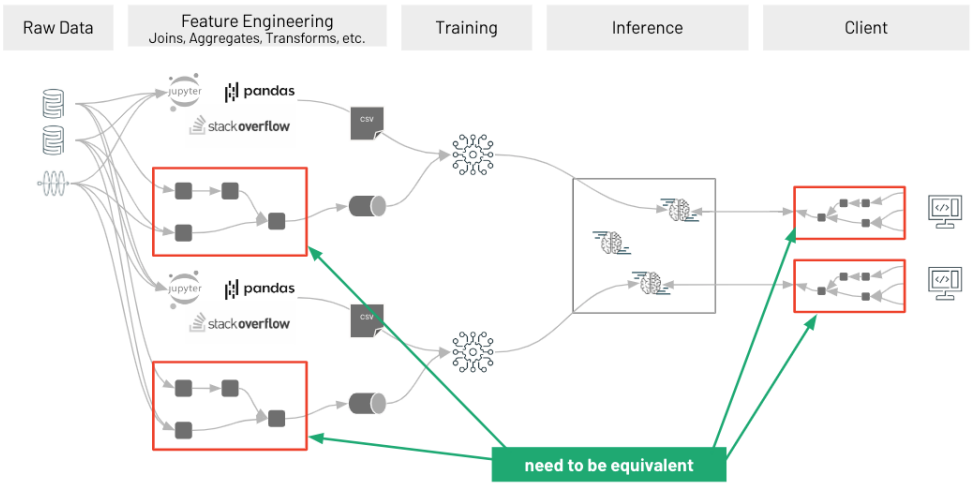


Figure 5.2 – You can avoid accuracy issues by guaranteeing that the transformations performed on training data are the same as those performed on inference data

We will prevent online/offline skew in the *Streaming Transactions* project by wrapping the data transformation processes into a packaged model in *Chapter 6*. Consistency can improve the accuracy of models, as it ensures the same input generates the same output in production as in training.